

Week 3: Library Desk Assistant

About the Session on Functions, Collections & Error Handling

Duration	90 minutes
Format	Pure Dart, individual work (no Flutter UI)
Tools	DartPad (dartpad.dev) or the Dart SDK (<code>dart run Week3.dart</code>)
Total marks	10 (+ up to 1 bonus)
Deliverable	<code>Week3.dart</code> and a screenshot of the console output

Learning Objectives

By the end of this lab you will be able to:

- Declare functions with positional, optional, named, `required` and default parameters
- Write arrow and anonymous functions, pass functions as arguments, and build closures
- Write a recursive function with a correct base case
- Use `List`, `Map` and `Set` with `map`, `where`, `reduce`, `fold`, `sort` and set operations
- Write simple generic classes and functions using `<T>`
- Handle errors with `try / on / catch / finally` and define custom exceptions
- Use `Future`, `async` and `await`

Scenario: Library Desk Assistant

You are building the back-end logic for a small campus library desk. Everything is **pure Dart** (no Flutter UI). Each part of the lab practises last lecture's topics on the same book data, so your code builds up into one working program.

Ground rules: work individually, run your program after every task, and compare your console output with the **Expected output** box. You may use DartPad or the Dart SDK. Write every function at the top level of the file (outside `partN()`), then call it from inside the matching `partN()` function.

Time Plan

Part	Topic	Time
Setup	Create the project, paste the starter code	5 min
Part 1	Functions & parameters	12 min
Part 2	Closures, higher-order functions & recursion	13 min
Part 3	List, Map & Set	20 min
Part 4	Generics	10 min
Part 5	Error handling & custom exceptions	15 min
Part 6	Future & async/await	7 min
Wrap-up	Format, screenshot, submit, reflect	8 min
Total		90 min

Setup (5 min)

1. Create a folder `Week3` and a file `Week3.dart` (or open `dartpad.dev`).
2. Paste the starter code below into the file.
3. Run `dart run Week3.dart`. You should see six headings printed, one per part.

```
// Week3.dart - Library Desk Assistant
// Name: _____ Roll no: _____

final List<Map<String, dynamic>> books = [
  {'title': 'Dart in Action', 'author': 'Ada', 'year': 2021,
   'copies': 3, 'tags': ['dart', 'programming']},
  {'title': 'Flutter Basics', 'author': 'Sam', 'year': 2023,
   'copies': 0, 'tags': ['flutter', 'mobile']},
  {'title': 'Clean Code', 'author': 'Martin', 'year': 2008,
   'copies': 2, 'tags': ['programming', 'design']},
  {'title': 'Algorithms', 'author': 'Knuth', 'year': 1968,
   'copies': 1, 'tags': ['programming', 'math']},
  {'title': 'UI Design', 'author': 'Nora', 'year': 2019,
   'copies': 4, 'tags': ['design', 'mobile']},
];

void main() async {
  part1();
  part2();
  part3();
  part4();
  part5();
  await part6();
}

void part1() { print('--- Part 1 ---'); }
void part2() { print('--- Part 2 ---'); }
void part3() { print('--- Part 3 ---'); }
void part4() { print('--- Part 4 ---'); }
void part5() { print('--- Part 5 ---'); }
Future<void> part6() async { print('--- Part 6 ---'); }
```

Tip: Put your code for each part inside its `partN()` function, below the heading `print`. Add new top-level functions and classes above `main()`.

Part 1: Functions & Parameters (12 min)

Topics: positional, optional positional, named and default parameters; arrow syntax

Task 1.1: Positional parameters

Write `double lateFee(int daysLate, double ratePerDay)` that returns the total late fee. Write it with **arrow syntax** (`=>`).

Task 1.2: Optional positional parameter

Write `String formatTitle(String title, [String? author])`. If `author` is `null` it returns just the title; otherwise it returns `'Title by Author'`. Use the `== null` check or `??`, not `!`.

Task 1.3: Named parameters with required and a default

Write `Map<String, dynamic> makeBook({required String title, required String author, int year = 2024, int copies = 1})` that returns a map with those four keys (in that order).

Task 1.4: Arrow function

Write `bool isClassic(int year)` that returns `true` when the year is before 2000. One line, arrow syntax.

Call all four inside `part1()` so the output matches:

EXPECTED OUTPUT

```
--- Part 1 ---
Late fee: 2.5
Dart in Action
Dart in Action by Ada
{title: Clean Code, author: Martin, year: 2024, copies: 1}
{title: Algorithms, author: Knuth, year: 1968, copies: 1}
true
false
```

Hints: use `lateFee(5, 0.5)`, `formatTitle('Dart in Action')`, `formatTitle('Dart in Action', 'Ada')`, `makeBook(title: 'Clean Code', author: 'Martin')`, `makeBook(title: 'Algorithms', author: 'Knuth', year: 1968)`, `isClassic(1968)` and `isClassic(2021)`.

Part 2: Closures, Higher-Order Functions & Recursion (13 min)

Topics: functions as objects, anonymous functions, closures, recursion

Task 2.1: Passing a function as an argument

Write `List<String> transformAll(List<String> items, String Function(String) fn)` that applies `fn` to every item and returns a **List**. Call it twice on `['Dart in Action', 'Clean Code']`: once with an anonymous function that upper-cases each string, and once with an arrow function that appends `'!'` to each string.

Task 2.2: A closure that remembers

Write `int Function() makeCounter()`. It declares `int count = 0;` and returns a function that increments and returns `count`. Create two counters `desk1` and `desk2`. Call `desk1()` three times and `desk2()` once, printing each result.

Task 2.3: A closure with a parameter

Write `double Function(int) makeFeeCalculator(double rate)` that returns a function taking `days` and returning `days * rate`. Create `studentFee` (rate 0.25) and `staffFee` (rate 0.10) and print the fee for 4 days for each.

Task 2.4: Recursion

Write a recursive function `int sumDigits(int n)`: if `n < 10` return `n`; otherwise return the last digit plus `sumDigits` of the remaining digits (use `%` and `~/`).

EXPECTED OUTPUT

```
--- Part 2 ---
[DART IN ACTION, CLEAN CODE]
[Dart in Action!, Clean Code!]
1
2
3
1
Student fee: 1.0
Staff fee: 0.4
Sum of digits: 8
```

Think: Why does `desk2()` print 1 even though `desk1` has already been called three times? What would happen to `sumDigits` if you deleted the base case?

Part 3: Collections: List, Map & Set (20 min)

Topics: map, where, reduce, fold, sort, Map lookups, Set operations, collection-for

Task 3.1: map and where

Using the `books` list, print (a) a `List` of all titles and (b) a `List` of titles of books that have at least one copy available. Remember that `map` and `where` return a lazy `Iterable`: finish with `.toList()`.

Task 3.2: reduce and fold

Use `fold` to compute the total number of copies across all books. Then map the books to a list of years and use `reduce` to find the oldest (smallest) year.

Task 3.3: Sorting without damaging the original

Make a `copy` of `books` (`List.of(books)` or `[...books]`), sort the copy by `'year'` ascending using `compareTo`, and print the titles in that order. `sort` changes a list in place, so the original books must stay untouched.

Task 3.4: Map

Write a top-level function `Map<String, int> buildStock()` that returns a map of title to copies using a `collection-for` inside a map literal. In `part3()`: print the map; use `forEach` to print a line for every title with zero copies; and print the copies of `'Unknown'` using `??` so a missing key gives `0`.

Task 3.5: Set

Build a `Set<String>` of every distinct tag used in `books` (a set literal with collection-for and the spread operator `...`). Then, given the two reading lists below, print their union, intersection and difference (a minus b).

```
var a = {'Dart in Action', 'Clean Code', 'Flutter Basics'};
var b = {'Clean Code', 'Flutter Basics', 'Algorithms'};
```

EXPECTED OUTPUT

```
--- Part 3 ---
Titles: [Dart in Action, Flutter Basics, Clean Code, Algorithms, UI Design]
Available: [Dart in Action, Clean Code, Algorithms, UI Design]
Total copies: 10
Oldest year: 1968
By year: [Algorithms, Clean Code, UI Design, Dart in Action, Flutter Basics]
Stock: {Dart in Action: 3, Flutter Basics: 0, Clean Code: 2, Algorithms: 1, UI Design: 4}
Out of stock: Flutter Basics
Copies of Unknown: 0
All tags: {dart, programming, flutter, mobile, design, math}
Union: {Dart in Action, Clean Code, Flutter Basics, Algorithms}
Common: {Clean Code, Flutter Basics}
Only in A: {Dart in Action}
```

Hints: values in the book maps are `dynamic`, so cast when you need a real type, e.g. `b['year']` as `int` or `b['tags']` as `List<String>`. For fold, start with `0`: `fold(0, (sum, b) => ...)`.

Part 4: Generics (10 min)

Topics: type parameters on classes and functions, compile-time type safety

Task 4.1: A generic class

Write `class Box<T>` with one field `T value` and a constructor `Box(this.value)`. Create a `Box<int>` holding 5 and a `Box<String>` holding 'dart' and print their values. Then temporarily try `intBox.value = 'hello'`; Read the compile error, then comment the line out.

Task 4.2: A generic function

Write `T firstOr<T>(List<T> items, T fallback)` that returns the first item, or `fallback` if the list is empty.

Task 4.3: A class with two type parameters

Write `class Pair<A, B>` with fields `first` and `second`. Override `toString()` so that printing a pair shows `(first, second)`.

EXPECTED OUTPUT

```
--- Part 4 ---
Box<int>: 5
Box<String>: dart
Dart in Action
z
(Dart in Action, 3)
```

Hints: call `firstOr(['Dart in Action', 'Clean Code'], 'none')`, `firstOr<String>([], 'z')` and `print(Pair('Dart in Action', 3))`.

Part 5: Error Handling (15 min)

Topics: custom exceptions, throw, try / on / catch / finally, `StateError`

Task 5.1: Custom exceptions

Write two classes that implements `Exception`, each with a `final String title` field set through the constructor: `BookNotFoundException` and `BookNotAvailableException`.

Task 5.2: Throwing

Write `void checkOut(Map<String, int> stock, String title)`. If `title` is not a key in `stock`, throw `BookNotFoundException`. If its count is 0 (or less), throw `BookNotAvailableException`. Otherwise decrease the count by one. (`stock[title]` is nullable, so you will need `!` here **after** you have already proven the key exists.)

Task 5.3: try / on / catch / finally

In `part5()`, create `var stock = buildStock();` and loop over `['Dart in Action', 'Flutter Basics', 'Unknown Book']`. For each title, inside a `try` call `checkOut` and then print `Checked out: <title>`. Handle each custom exception with its own `on` clause, and add a `finally` that prints `Transaction logged`. Finally, print the copies left of `'Dart in Action'`.

Task 5.4: A built-in exception

Write `Map<String, dynamic> findBook(String title)` using `books.firstWhere(...)`. Call `findBook('Missing')` inside a `try` and catch the `StateError` it throws, printing `Search failed: no such book`.

EXPECTED OUTPUT

```
--- Part 5 ---
Checked out: Dart in Action
Transaction logged.
Sorry: "Flutter Basics" has no copies left
Transaction logged.
Not found: "Unknown Book"
Transaction logged.
Copies left of Dart in Action: 2
Search failed: no such book
```

Watch out: if `checkOut` throws, the line after it inside the `try` is skipped. That is exactly why `Checked out:` is printed after the call. Also, put specific `on` clauses **before** any general catch (`e`).

Part 6: Future & async/await (7 min)

Topics: `Future<T>`, `async`, `await`, errors in `async` code

Task 6.1: Await a Future

Write `Future<String> fetchBookOfTheDay() async` that waits one second with `await Future.delayed(Duration(seconds: 1))`; and then returns `'Dart in Action'`. In `part6()` print `Fetching...`, `await` the result, and print `Book of the day: ....`

Task 6.2: See what happens without await

Temporarily remove `await` and print the result. Note what is printed (something like `Instance of '_Future<String>'`); can you figure out the reason (if not will be discussed again in next lecture), put `await` back.

Task 6.3: Errors in async code

Write `Future<String> fetchBroken() async` that waits 500 ms and then does `throw Exception('Server down');`. Call it inside `part6()` with `try / catch` and print `Fetch failed: $e`.

EXPECTED OUTPUT

```
--- Part 6 ---
Fetching...
Book of the day: Dart in Action
Fetch failed: Exception: Server down
```

Wrap-up & Submission (8 min)

1. Run `dart format .` and `dart analyze` (or DartPad's Format button). Fix any warnings.
2. Take a screenshot of your full console output.
3. Submit `Week3.dart` and the screenshot by pushing to your course GitHub repository.

Answer these reflection questions in comments at the bottom of `Week3.dart` (one or two sentences each):

1. When would you choose `fold` over `reduce`?
2. What does it mean that a closure "captures" a variable? Which variable was captured in `makeCounter`?
3. Why must `BookNotAvailableException` come before a general `catch (e)`?
4. Why does forgetting `await` still compile, but give the wrong result?

Common Pitfalls

- Forgetting `.toList()` after `map()` / `where()`: you get an `Iterable` printed with parentheses.
- Calling `reduce()` on an empty list throws; `fold()` with a starting value is safe.
- Mutating a list while looping over it with `for-in` throws `ConcurrentModificationError`.
- Catching everything with a bare `catch (e)` can hide real bugs.
- Using `!` to silence null errors instead of checking the value first.

Bonus Challenges (up to 1 extra mark)

- **B1:** Build `Map<String, List<String>>` that groups titles by tag, e.g. `programming` maps to its three titles.
- **B2:** Write a generic `List<T> filterBy<T>(List<T> items, bool Function(T) test)` and reimplement the "available books" query from Task 3.1 with it.
- **B3:** Start two `Futures` (for example two `fetchBookOfTheDay()` calls) and run them together with `Future.wait`. Measure the total time: it should be about 1 second, not 2.

Marking Scheme

Section	Marks	What is checked
Part 1: Functions	1.5	Correct parameter kinds, arrow syntax, output matches
Part 2: Closures & recursion	1.5	Independent counters, correct base case, function arguments
Part 3: Collections	2.5	map/where/toList, fold/reduce, non-destructive sort, Map, Set operations
Part 4: Generics	1.0	Box<T>, generic function, Pair<A,B> with toString
Part 5: Error handling	2.0	Custom exceptions, on-clauses order, finally, StateError
Part 6: Async	1.0	await used correctly, error caught
Code quality & reflection	0.5	Formatted, no analyzer warnings, reflection answers
Total	10	